

Arquitectura ELK Multi-fuente (Batch y Near Real-Time)

1. Objetivo del Proyecto

Diseñar e implementar un flujo de ingesta y visualización de datos utilizando el stack ELK (Elasticsearch, Logstash, Kibana), combinando paradigmas de procesamiento Batch y Near Real-Time (NRT). El caso de uso se centra en el análisis operativo, integrando datos de clientes, productos, reseñas y telemetría espacial de pedidos.

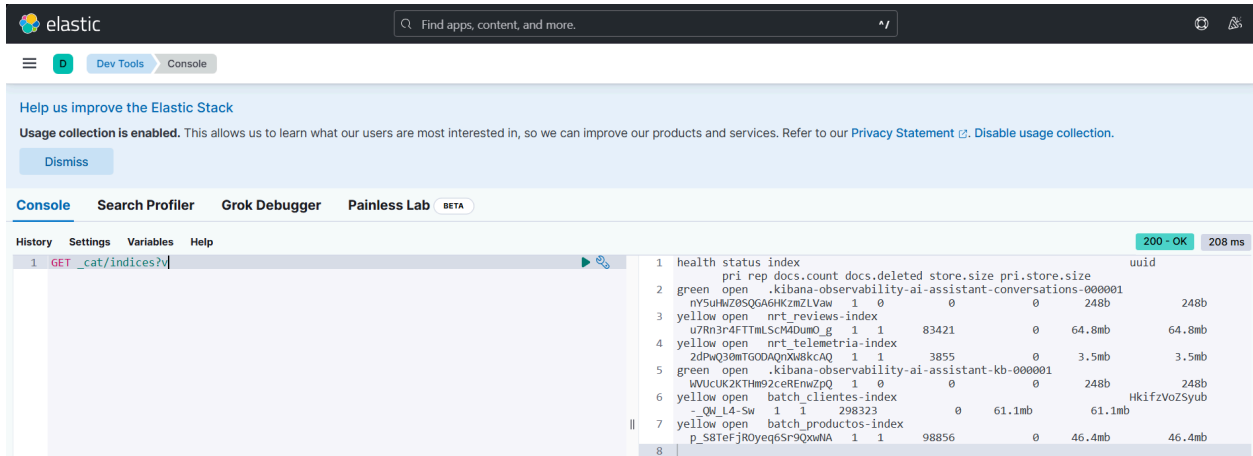
2. Arquitectura Diseñada e Implementada

La arquitectura se desplegó en un entorno contenerizado utilizando Docker.

☐		Name ↑	Container ID	Image	Port(s)	CPU (%)	Actions
☐	▼	taller- elk-multisource	-	-	-	8.42%	
☐		elasticsearch_taller	4ab78e797e39	elasticsearch	9200:9200	2.24%	
☐		kibana_taller	23e87f4c3b3c	kibana/kibana	5601:5601	0.95%	
☐		logstash_taller	f4fb8f716576	logstash/logstash		5.23%	
☐		postgres_taller	4f5cb0f89633	postgres	5432:5432	0%	

Logstash actúa como el motor central de ingesta, normalización y enrutamiento hacia Elasticsearch, desde donde Kibana consume los datos para el análisis visual.

Fuente de Datos	Tipo de Flujo	Tecnología / Origen	Índice en Elasticsearch
Catálogo de Productos	Batch	Archivo estático (CSV)	batch_productos-index
Base de Clientes	Batch	Base de datos PostgreSQL (JDBC)	batch_clientes-index
Satisfacción (Reseñas)	Near Real-Time (NRT)	Flujo de logs de texto (CSV stream)	nrt_reviews-index
Telemetría de Pedidos	Near Real-Time (NRT)	Flujo JSON vía Python	nrt_telemetria-index



2.1 Orquestación e Infraestructura Base

El entorno completo fue aprovisionado mediante Docker Compose, desplegando cuatro servicios interconectados a través de una red virtual dedicada (e1k_network). Esta segmentación garantiza la resolución de nombres por DNS interno y aísla el tráfico de ingesta. Los contenedores incluyen Elasticsearch (almacenamiento e indexación), Kibana (interfaz analítica), Logstash (motor ETL) y PostgreSQL (repositorio relacional). Se utilizaron volúmenes persistentes locales (es_data y postgres_data) para salvaguardar el estado del clúster y de las bases de datos ante reinicios del ciclo de vida de los contenedores.

2.2 Flujos de Procesamiento Batch (Cargas Históricas)

El procesamiento por lotes se configuró en Logstash para consolidar los catálogos estáticos y la información dimensional del ecosistema Olist.

- **Catálogo de Productos (CSV estático):** Logstash procesa un archivo plano directamente desde un volumen mapeado (/usr/share/logstash/my_data/). Se implementó un filtro CSV en el pipeline para estructurar los campos (ID, categoría, dimensiones, peso) y un bloque mutate para transformar tipos de datos a enteros. Los eventos resultantes se enrutan al índice batch_productos-index.
- **Base de Clientes (PostgreSQL vía JDBC):** Los datos maestros de clientes residen en el contenedor de PostgreSQL 15. Durante el despliegue inicial, el script init.sql inicializa y puebla la tabla customers. El pipeline de Logstash utiliza el plugin **JDBC** con el controlador oficial (postgresql-42.6.2.jar) para autenticarse, ejecutar una consulta SQL de extracción (SELECT customer_id, customer_zip_code_prefix, customer_city, customer_state FROM customers) y transferir los registros hacia el índice batch_clientes-index.

2.3 Flujos de Procesamiento Near Real-Time (NRT)

Para simular el comportamiento transaccional en vivo, se implementaron dos flujos continuos que inyectan datos operativos en la arquitectura.

- **Telemetría Espacial de Pedidos (JSON):** Un script iterativo en Python

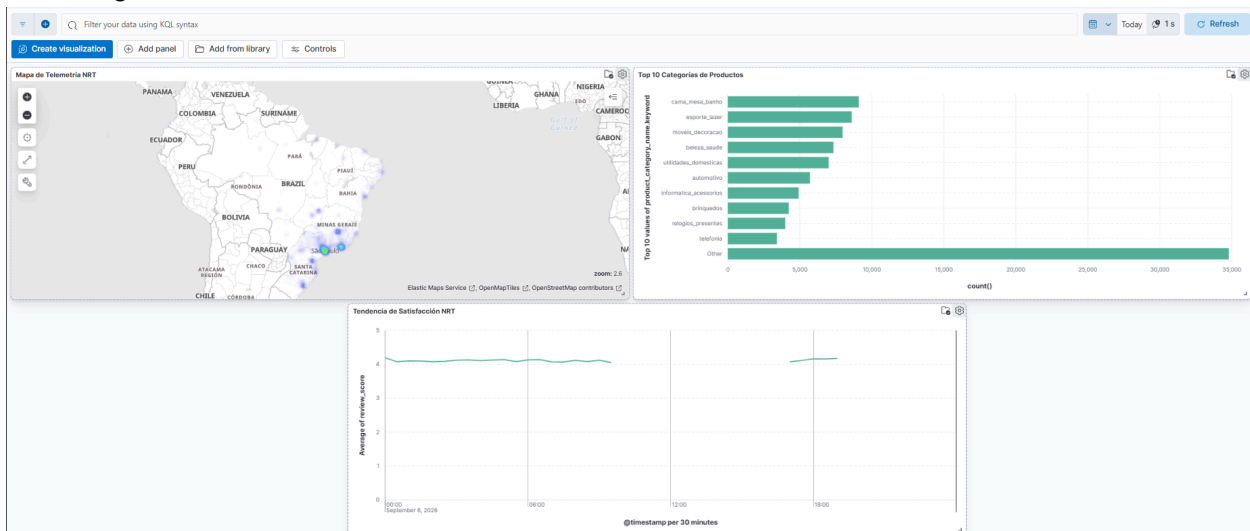
(start_nrt.py) actúa como generador simulando la creación de nuevas órdenes. Emite *payloads* en formato JSON que contienen el ID del pedido, estado, y coordenadas (latitud/longitud). Logstash lee este flujo continuo y lo dirige al índice `nrt_telemetria-index`. Previo a la ingesta, se configuró un `Index Template` en Elasticsearch vía API para forzar que el campo de coordenadas asuma estrictamente el tipo `geo_point`.

- **Monitor de Satisfacción (Reseñas Textuales):** Las reseñas de clientes operan como un flujo NRT *append-only*. A medida que el script adjunta nuevas líneas al log de reseñas, Logstash las ingiere de forma inmediata. Se configuró el filtro CSV con el parámetro `quote_char => '"'` para aislar correctamente las cadenas de texto (comentarios largos con comas internas), garantizando que el campo `review_score` se indexe limpiamente en `nrt_reviews-index` para su monitoreo en vivo.

3. Diseño del Dashboard y Data Storytelling

La construcción del dashboard en Kibana se realizó aplicando rigurosos principios de storytelling con datos y los criterios de visualización del modelo de Tamara Munzner. El layout se estructuró jerárquicamente para guiar la atención del usuario sin sobrecarga cognitiva:

- **Visión Macro (Posición y Espacio):** Un mapa interactivo (Capa de Clusters y Documentos) ubicado en la parte superior, mostrando la densidad de pedidos entrantes en tiempo real.
- **Análisis Categórico (Longitud de marca):** Un gráfico de barras horizontales mostrando el Top 10 de categorías de productos, optimizado para la lectura rápida de variables nominales extensas.
- **Tendencia Temporal (Conexión):** Un gráfico de línea monitoreando el promedio de calificación de reseñas a lo largo del tiempo, actualizado automáticamente cada 5 segundos mediante el motor NRT.

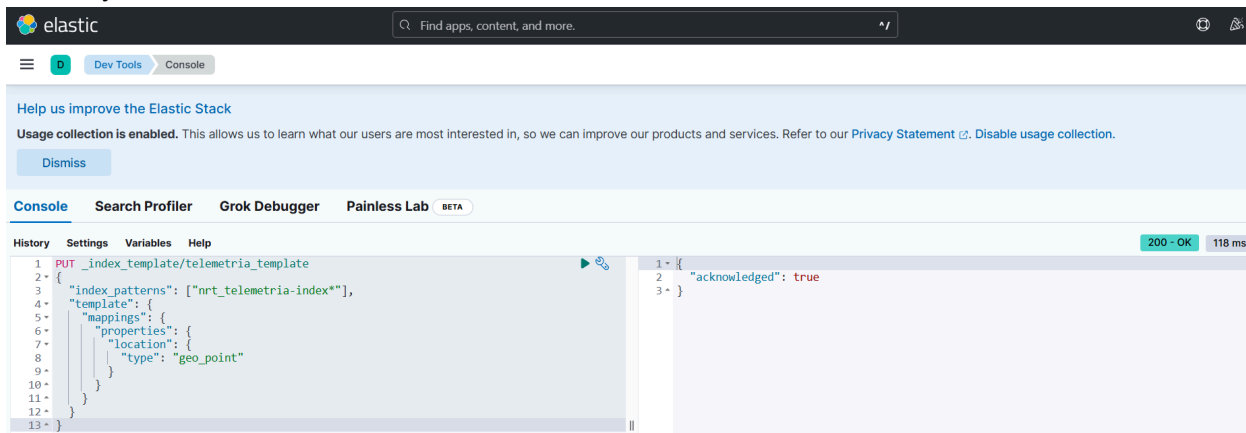


4. Retos Encontrados y Soluciones Implementadas

Durante la implementación y despliegue del entorno, se presentaron diversos desafíos técnicos que fueron resueltos mediante ajustes en la infraestructura y configuración:

1. Mapeo de Coordenadas Geográficas en Kibana Maps:

- *Reto:* Kibana no reconocía las coordenadas de telemetría espacial, ya que Elasticsearch las indexó por defecto como números independientes (lat/lon) en lugar de un objeto geoespacial, impidiendo la creación del mapa.
- *Solución:* Se reestructuraron los datos eliminando el índice mal formado y aplicando un Index Template vía Dev Tools. Esto forzó el mapeo del campo location bajo el tipo geo_point antes de reanudar la inyección en tiempo real desde Python.



2. Persistencia y Autenticación de JDBC en PostgreSQL:

- *Reto:* El contenedor de PostgreSQL no ejecutaba el script de inicialización (init.sql) y Logstash generaba errores FATAL de autenticación al intentar conectar. Esto ocurrió por discrepancias de credenciales arrastradas en volúmenes huérfanos de Docker.
- *Solución:* Se aplicó una purga de los volúmenes de almacenamiento locales (docker-compose down -v), garantizando un inicio en limpio donde el motor de la base de datos levantó las tablas correctamente y Logstash logró extraer el volumen histórico.

3. Errores de Parsing en Textos Libres (Logs NRT):

- *Reto:* El flujo de reseñas registraba etiquetas _csvparsefailure debido a que los comentarios de texto de los usuarios contenían comas internas, rompiendo la delimitación estándar de las columnas CSV.
- *Solución:* Se ajustó el pipeline de Logstash incorporando el parámetro quote_char => '"' dentro del plugin CSV, encapsulando adecuadamente las cadenas de texto y permitiendo que campos críticos como review_score se convirtieran a enteros de forma limpia.

5. Conclusión

El despliegue de esta arquitectura validó la robustez de Elasticsearch y Logstash para orquestar la convergencia de modelos analíticos dispares (históricos y en vivo). La superación

de retos de tipificación dinámica y ruteo espacial demostró que la configuración del esquema a nivel de base de datos es tan crítica como el diseño visual en la capa de usuario. El dashboard final cumple exitosamente con el objetivo de presentar un centro de control unificado, responsivo y orientado a la toma de decisiones.